VeriRefactor: Verified, Agentic Test-Smell Refactoring with Kill-Set Gating and Detector Consensus

M Saqib Atif, Gulsher Khan, Hasan Altaf
Group G2-10, AI4015, FAST-NUCES Islamabad
i230769@isb.nu.edu.pk, i230066@isb.nu.edu.pk, i230698@isb.nu.edu.pk
IDs: 23I-0769, 23I-0066, 23I-0698

***Abstract*—Large language models can refactor smelly unit tests, but a refactoring that compiles and passes may still check less than the original. UTRefactor, the state of the art, accepts any output that compiles and passes and relies on a single smell detector. We present VeriRefactor, a multi-agent system in which tools decide acceptance: a Supervisor coordinates Detection, Planner, Refactor and Verifier agents and chooses its next step from the Verifier's report. Three components target three gaps: a mutation-testing gate rejects any refactoring that loses a mutant the original test killed (G1); smells are targeted and counted as removed only when tsDetect and JNose agree (G2); and adaptive supervision with memory replaces a fixed retry script (G6). On 150 held-out tests from Commons-cli, Gson and Jsoup, with `gpt-oss-120b` and three seeds, the full system accepts 52.4 $\pm$ 1.0% of the tests and no accepted output loses a mutant; with the gate switched off, 8.0 $\pm$ 0.5% of accepted outputs do (exact McNemar, $p < 0.001$). The two detectors disagree sharply on some smells ($\kappa = 0.02$ for ExceptionHandling), but targeting only the smells both confirm lowered the accepted-and-safe rate by 15 points, and the adaptive Supervisor did not beat a fixed retry script (24.0% against 35.3% accepted and safe, seed 1). Replaying UTRefactor's published outputs through our verifier reproduces its Commons-cli result (86.4% against 86% smell reduction) but not its Gson and Jsoup results. We release the system, all trajectories and the scripts that rebuild every table.**

I. Introduction

LLM refactoring of test smells can silently weaken a test: the new code compiles and passes but kills fewer mutants. UTRefactor [1] accepts a refactoring once it compiles and passes. We study three gaps in that setting. **RQ1 (G1)**: does a kill-set gate stop accepted refactorings that lose mutants? **RQ2 (G2)**: does requiring two detectors (tsDetect, JNose) to agree make smell removal more trustworthy? **RQ3 (G6)**: does adaptive supervision (retry, switch strategy, memory) beat a fixed retry script and a single tool-using agent? Contributions: (i) a verifier that accepts a refactoring only if it compiles, passes, removes the targeted smells and preserves the PIT kill set; (ii) tsDetect+JNose consensus targets; (iii) an adaptive Supervisor with an LLM planner and cross-test memory; (iv) a controlled comparison of six systems and five ablations on 150 held-out tests, with three seeds for the full system and its gate ablation, and with every trajectory released.

II. Related Work

Test smells. Test smells [2], [3] are design problems in test code whose harm is debated [4]; tsDetect [5] and JNose [6] detect them, and their agreement is measured with Cohen's κ [7], [8]. UTRefactor [1] refactors smells with an LLM and a knowledge base but accepts any output that compiles and passes. *Mutation testing and oracles.* Mutants are a reasonable proxy for real faults [9]; PIT [10] computes kill matrices [11], [12], and the oracle problem [13] motivates preserving the kill-set rather than trusting passing tests. Generated tests [14], [15] face the same weakness. *LLM agents.* Reasoning and acting [16], self-feedback [17], verbal memory [18], chain-of-thought [19], tool use [20] and repository agents [21], [22] on code models [23] motivate our Supervisor, but none gates acceptance on mutation evidence. *Acceptance by execution.* Repository-level benchmarks such as SWE-bench [21] judge an agent's patch by running tests, and agent designs such as SWE-agent [22] and Reflexion [18] feed execution results back into the next attempt. Test refactoring needs a stronger oracle than "the tests pass", because the code under change is the test itself; a refactored test that passes may simply check less [13]. Mutation analysis supplies that oracle at the cost of runtime [11], [12], which is why we use it as a gate and report its cost separately. *Statistics.* We use exact McNemar [24], Wilcoxon [25] and Wilson intervals [26].

III. Problem Formulation

Input: a Java/JUnit project P and a test suite $T = \{t_1, \ldots, t_n\}$ (test methods). Smell set S (the tsDetect smells in [1]). Tool set $\mathcal{K}$: for the single-agent baseline b2, $\{\mathsf{detect}, \mathsf{compile_and_run}, \mathsf{pit}, \mathsf{submit}\}$; s3 calls the same tools through its Verifier as one verification step. For a test t, $D(t) \subseteq S$ is its detected smells. A state is $(t, t', D(t'), a)$: original, current candidate, its smells, and the attempt count $a \leq a_{\max}$ (with a per-test token cap). The output is a decision $d(t) \in \{\text{accept}, \text{reject}, \text{escalate}\}$ and, if accepted, t'. Constraints: t' compiles and passes; $D(t') \cap D(t) = \emptyset$ for targeted smells; behaviour preservation $K(t') \supseteq K(t)$ where K is the set of PIT mutants killed. Objective: maximise the rate of accepted-and-safe tests at minimal tokens.

TABLE I
AGENTS, TOOLS AND WHERE EACH GAP PLUGS IN.

Agent	Tools	Gap
Detection	tsDetect, JNose	G2 (consensus targets)
Planner	LLM (JSON plan), KB rule	–
Refactor	LLM	–
Verifier	javac/JUnit, tsDetect, JNose, PIT	G1 (kill-set gate)
Supervisor	memory	G6 (adaptive policy)

IV. PROPOSED SYSTEM

Figure 1 shows the system. A Supervisor drives four agents: Detection (tsDetect, JNose, G2 consensus), Planner (LLM-written JSON plan with a rule fallback), Refactor (LLM) and Verifier (patch, compile, run, smells, PIT kill-set gate, G1). The Supervisor accepts, retries, switches strategy, narrows targets, reverts to the best attempt or escalates (G6); memory holds failed proposals within a test and failed approaches across tests. Every step is logged as a trajectory event.

Memory. Within a test, the Supervisor stores hashes of failed proposals (never re-verified) and the (strategy, gate) pairs that failed; across tests it stores, per project and smell, the planned approaches that failed, and shows them to the Planner.

Supervisor policy. After every verification the Supervisor picks one of six actions. It *accepts* when all gates pass and no targeted smell is left. It *retries* with the error text the first time a strategy fails at a given gate, and *switches* strategy (LLM plan, then the knowledge-base rule, then a plain prompt, then a re-plan that includes the failure and the cross-test memory) when the same strategy fails at the same gate again or the reply repeats a rejected proposal. When every gate passes but some targeted smells remain, it *narrows* the targets to those smells; when an attempt gets less far through the gates than an earlier one, it *reverts* so that the next retry starts from the best proposal so far. When attempts or the per-test token budget run out, it *escalates*: the test is recorded as escalated and placed in a review queue, where a person can accept or reject it; these human decisions never change the automatic metrics. The -G6 ablation replaces this policy with b1's script: one knowledge-base prompt, then up to two retries that carry the error text, then rejection.

Verifier. The Verifier runs the same stages for every system. Static checks come first: the reply must parse, keep the test's assertions (a refactoring that drops all assertions or leaves an empty test is rejected), have balanced braces, and differ from every proposal already rejected for this test. The patch is then applied to the test class; the class is compiled once with `javac` against the module classpath recorded by Maven and run with the JUnit launcher, falling back to a full Maven run when the fast path fails on the original class. Both detectors analyse the refactored methods, and PIT computes the kill set of the new tests, which is compared with the original's. Whatever happens, the original file is restored, and a `git status` check after each test stops the run if a checkout was left modified.

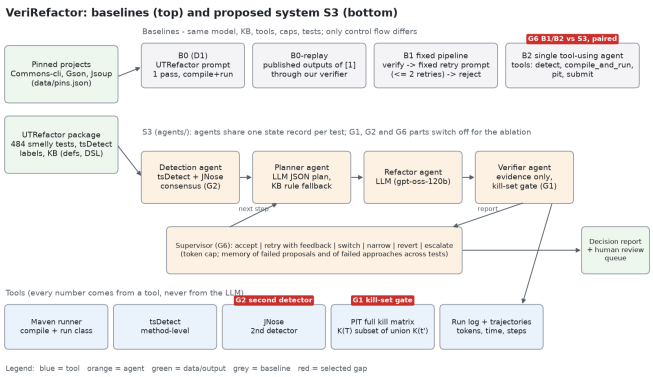


Fig. 1. Architecture and where each gap plugs in.

V. EXPERIMENTAL SETUP

Benchmark. We use the test methods of the UTRefactor replication package [1] (Apache Commons-cli, Gson, Jsoup). The package has reports for 492 test methods; 484 match the project commits we pinned verbatim and form our evaluation set (Commons-cli 77, Gson 238, Jsoup 169; the pinned commits contain 97.9–98.8% of the published inputs verbatim). Projects are built at the pinned commit with Maven 3.9.9 (each project with its pinned JDK 17 where its build needs it; JNose and the tool chain run on JDK 21); Gson is evaluated against its own test dependencies at that commit.

Splits. The 484 tests are split once, stratified by project and single/multi-smell, with a fixed seed (commit `0e1f7da`, `data/splits/`): *dev* 80 tests (13/39/28 across Commons-cli/Gson/Jsoup), *test* 150 (24/74/52) and *rest* 254 (40/125/89). Prompts, thresholds, attempt and token caps were tuned on dev only. All reported LLM runs use the test split and the runner's `--frozen` guard, which refuses to start from a dirty tree and records the git commit. The rest split is used only for runs without an LLM (E1, E4 detector agreement).

Systems. E1 is UTRefactor's published outputs measured with our verifier (b0replay, no LLM). E2 compares: b0, UTRefactor's prompt and knowledge base with our model, one pass; b1, the same prompt followed by a fixed retry script (at most 2 retries with the error text); single, a plain prompt without the knowledge base; b2, a single tool-using agent with the same tools and token cap as our system; and s3, our multi-agent system. All systems share the model, knowledge base, tools, per-test token cap and test set. E3 is s3 with all three gap components, E5 switches one off at a time (-G1: kill-set measured, never rejects; -G2: tsDetect only; -G6: fixed retry policy; and memory off). E2 numbers are labelled *reproduced* when we ran the system and *paper-reported* when they are copied from [1].

Model and parameters. All LLM calls go to Cerebras `gpt-oss-120b` (free tier, 5 requests/min, 950k tokens/day) at temperature 0.7, at most 8000 output tokens per call and 60 000 tokens per test. Because one test-split run of the full system takes hours (PIT), we use a reduced grid: seeds 1, 2 and 3 for the full system (E3) and for -G1 (E5),

and seed 1 for the baselines and the other ablations (see `experiments/E*.json`); mean ± sd is therefore reported only where 3 seeds were run. A sensitivity run (ES) varies the retry cap (0, 1, 2, 4) for s3 on the first 20 dev tests; the LLM response cache is keyed by prompt and seed. Every run records the model ID, temperature, system-prompt hash, seed and git commit in `config.json` and in the trajectory metadata.

Tooling. tsDetect (v3 jar from the replication package) and JNose (jnose-core 0.9.0, the last release built for Java 21) detect smells at method level; PIT 1.17.4 computes the full mutation matrix; the fast runner compiles once per Maven module and runs each test class through the JUnit launcher. The reported runs used an HP Omen 16 laptop with Ubuntu 24.04 under WSL2 (32 logical CPUs and 15 GB of memory visible to Linux), Python 3.12, OpenJDK 17 for the project builds, JDK 21 for JNose and Maven 3.9.9. The pipeline restores every touched file after each test, and a post-test `git status` check stops the run if a checkout was left modified.

Execution of the reported runs. All 19 test-split and dev runs were made from commit `3b992f2` with the `--frozen` guard, which refuses to start when a tracked file differs from the commit and records the commit in each run's `config.json`. To fit the free tier, configurations ran in parallel, each in its own copy of the repository with its own API key, so no two runs ever shared a key or a working directory. A run that hit a key's daily limit was resumed with another key; resumption skips the tests whose results are already saved and redoes only the test in progress. With up to eight runs on one machine, PIT used 4 to 8 threads per run. This affects wall-clock times (Table VII); outcomes could change only through PIT's timeout rule, since a slower run can turn a surviving mutant into a timed-out, and therefore killed, one.

VI. Evaluation Methodology

Let T be the evaluated tests, $A \subseteq T$ the accepted ones, and for $t \in T$ let $D(t)$ be its targeted smells (the smells labelled in [1] that the detector in use confirms) and $D'(t)$ the smells still reported in its refactored methods. Rates are percentages; the accepted-losing-mutants rate carries a Wilson 95% interval.

Accepted rate $= |A|/|T|$. For b1, b2 and s3 a test is accepted only if the refactoring is statically sane (no dropped or empty test), compiles, passes, and removes the targeted smells; with G1 on it must also preserve the kill-set. single, b0 and b0replay follow [1]: accepted if the output compiles and passes (the kill-set is only measured). **Compile rate** and **pass rate** are the share of tests whose *last* verification report compiled, respectively passed.

Smell reduction has three definitions. Rule i (tsDetect, as in [1]): $\sum_t |D(t) \setminus D'_{ts}(t)| \,/\, \sum_t |D(t)|$ over all tests; we also report it over accepted tests only. Rule ii (strict): a smell counts as removed only if both tsDetect and JNose no longer report it. Rule iii (manual) is defined in `labelling_guide.md`: two labellers judge a 100-test sample independently and Cohen's κ measures their agreement. We did not complete this sample before the deadline, so rule iii is defined but not reported.

Behaviour preservation. For test t let $K(t)$ be the set of PIT mutants killed by t, from the full mutation matrix. The original's $K(t)$ is computed on the unmodified class before the patch is written. A refactoring that yields tests $t'_1..t'_m$ preserves behaviour if $K(t) \subseteq \bigcup_i K(t'_i)$. **Accepted-losing-mutants rate** is the share of accepted tests violating this; **accepted-and-safe rate** is the share of T that is accepted, preserves K, and leaves none of the smells labelled in [1] under rule ii. The last condition uses all labelled smells, not only the targeted ones, so a system that targets fewer smells is not rewarded for leaving the others in place. Tests with an empty baseline $K(t)$ are counted as *unknown*, not as preserved.

Efficiency and reliability. Tokens per test (total, and split into prompt and completion from the provider's usage field), tokens per accepted test, wall-clock latency per test (includes rate-limit waits), tool calls and retries (reported as totals and per-test means), and **recovery rate** = accepted tests among those that needed at least one retry. Cost is $0 on the free tier; we report tokens instead.

Comparison protocol. Every system runs the same 150 test-split tests. The full system and its -G1 variant run with seeds 1, 2 and 3 and are reported as mean ± sd; the baselines and the other ablations run once (seed 1), because a test-split run with PIT takes several hours. Paired comparisons on the same tests (pooled over seeds where both systems have them) use the exact McNemar test (binary outcomes) and the Wilcoxon signed-rank test (continuous), with Holm correction across the comparisons in a table and bootstrap confidence intervals for differences. **Metric-definition differences from [1]:** (i) our evaluation set has 484 tests (the replication package's reports cover 492 and Table 2 of [1] states 593), because the paper does not name its project commits and we pin the commits that contain most of its inputs verbatim; (ii) we accept an output only if it compiles, passes, removes the targeted smell and (G1) kills every mutant the original killed, whereas rule i alone credits any output that no longer shows the smell; (iii) smells are detected per test method; (iv) Gson is built with its own dependencies. Replaying the published outputs through our verifier (`b0replay`) matches the paper on Commons-cli (86.4% against 86%) but not on Gson (57.9% against 89%) or Jsoup (46.0% against 91%), Table III; we report these as measured with our build and counting, not as a reproduction of the paper's figures.

VII. Results

All numbers in this section refer to the 150-test split, with the same tests for every system. Where three seeds were run (s3 and s3-G1) we give mean ± standard deviation; every other row is a single seed. Paired tests against a single-seed configuration use seed 1 of s3 (51.3% accepted, 24.0% accepted-and-safe; the three-seed means are 52.4 ± 1.0% and 25.8 ± 1.6%). Table II lists all systems. Table III sets our numbers next to the ones printed in [1], and each base-paper figure is marked as *reproduced* (we ran it) or *paper-reported* (copied from [1]).

TABLE II

ALL SYSTEMS ON THE TEST SPLIT (E1–E3). RED. I: SMELL REDUCTION COUNTED BY TSDETECT ALONE, AS IN [1]; RED. II: A SMELL COUNTS AS REMOVED ONLY WHEN BOTH DETECTORS STOP REPORTING IT; LOSE MUT.: SHARE OF ACCEPTED OUTPUTS THAT LOSE AT LEAST ONE MUTANT; ACC.+SAFE: ACCEPTED, KILL SET PRESERVED, AND NONE OF THE LABELLED SMELLS LEFT UNDER RULE II. TOP ROW: MEAN ± SD OVER THREE SEEDS.

System	Accepted %	Passes %	Red. i %	Red. ii %	Lose mut. %	Acc.+safe %	Tok/test
s3	52.43 ± 1.03	74.9 ± 2.03	46.07 ± 1.6	52.47 ± 1.7	0	25.8 ± 1.56	9182.33 ± 145.89

System	Accepted %	Passes %	Red. i %	Red. ii %	Lose mut. %	Acc.+safe %	Tok/test
b0	54.7	54.7	64.2	61.7	1.2	26.7	3183
single	82	82	53.8	49.1	–	–	1799
b1	41.3	67.3	63	60.5	0	35.3	8495
b2	26.7	43.3	60.7	61.8	0	24	4121

TABLE III

RULE-I SMELL REDUCTION (%) PER PROJECT: PAPER-REPORTED [1] AGAINST OUR REPRODUCTIONS. B0REPLAY FEEDS THE PUBLISHED REFACTORINGS THROUGH OUR VERIFIER; B0 RUNS UTREFACTOR'S PROMPT WITH OUR MODEL.

Project	[1] paper-reported	b0replay (reproduced)	b0 (reproduced)
commons-cli	86	86.4	97.5
gson	89	57.9	67.2
jsoup	91	46	48.8

Base-paper comparison (E1, E2). On Commons-cli our replay of the published outputs reaches 86.4% rule-i reduction against the 86% reported in [1], so the counting matches where the build matches. On Gson (57.9% against 89%) and Jsoup (46.0% against 91%) it does not. Most of the Gson gap comes from outputs that never compile against the project's own test dependencies: 131 of the 238 published Gson refactorings call `assertDoesNotThrow`, which JUnit 4 does not have, and about 61 pass a message argument to a Truth assertion. Across all three projects only 28.4% of the published outputs compile in our setup and 18.4% pass, so b0replay ends at 18.4% accepted and 11.8% accepted-and-safe. Running UTRefactor's prompt with our model (b0) gives 54.7% accepted at 3,183 tokens per test; the plain prompt without the knowledge base (single) is accepted more often (82.0%) because it is judged by compiling and passing only, but it removes fewer smells (53.8% under rule i, 49.1% under rule ii). We read these numbers as a statement about our build, not as evidence that [1] miscounted: the package does not document the dependencies it compiled against.

A. RQ1 (G1): does the kill-set gate stop weakened tests?

H1: with the gate off, more accepted refactorings lose a mutant, while acceptance changes little. The answer is yes. With the gate, no accepted output of s3 lost a mutant in any of the three seeds. With PIT still measuring but the gate unable to reject (s3-G1), 8.0 ± 0.5% of accepted outputs lost at least one mutant, and acceptance rose by only 2.7 points (55.1 ± 1.4% against 52.4 ± 1.0%). On the tests with a non-empty baseline kill set (129 to 134 tests per seed), the loss rate is 0% against 4.8 ± 0.6%. Pooled over 450 (test, seed) pairs, 19 pairs were accepted-and-weakened without the gate and none with it (exact McNemar, Holm-corrected $p < 0.001$; 95% bootstrap interval for the difference -6.2 to -2.4 points). Token use did not differ (Wilcoxon $p = 0.37$). The baselines also lose mutants: 3 of the 88 checked published outputs (3.4%, Wilson interval 1.2–9.5%) and 1 of 81 b0 outputs (1.2%). These are outputs that [1]'s acceptance rule would count as successful refactorings.

B. RQ2 (G2): does detector consensus make smell removal more trustworthy?

H2: the gap between rule i and rule ii is largest for the smells on which the detectors disagree. On all 484 originals, JNose confirms 76.2% of the 1,086 smell labels of [1]. The two tools agree well on most smell types (κ between 0.68 and 1.0) and badly on ExceptionHandling (tsDetect 224 tests, JNose 10, $\kappa = 0.02$) and DuplicateAssert (45 against 130, $\kappa = 0.42$); JNose flags only `try` blocks, while tsDetect also flags `throws` clauses. In the s3 outputs the two rules differ for two smell types: AssertionRoulette (71.2% removed under rule i, 63.0% under rule ii, 73 targeted) and MagicNumberTest (60.0% against 53.3%, 15 targeted). For every other type the rules agree, so H2 holds only weakly. The consensus component itself did not help. Targeting only the smells both tools confirm made s3 accept more tests than s3-G2, which targets every tsDetect smell (51.3% against 44.7%, $p = 0.09$, not significant after correction), but it lowered accepted-and-safe from 39.3% to 24.0% (Holm $p < 0.001$). The reason is in the metric: accepted-and-safe requires every labelled smell to be gone under rule ii, and smells that only tsDetect reports are never targeted under consensus, so they stay in the output. Against b0, the share of tests with all labelled smells gone under rule ii is 27.3% for s3 and 42.0% for b0 (Holm $p = 0.002$).

C. RQ3 (G6): does adaptive supervision beat fixed retries and a single agent?

H3: on multi-smell tests and on hard tests (those b0 rejects), s3 reaches a higher accepted-and-safe rate than b1 and b2 at equal or lower token cost. H3 is not supported. On the 117 multi-smell tests s3 reaches 14.0 ± 0.5% accepted-and-safe, against 28.2% for b1 and 17.1% for b2; on the 68 hard tests the figures are 6.4 ± 3.1%, 8.8% and 0.0%. Over all 150 tests, s3 is below the fixed-retry pipeline (24.0% against 35.3%, Holm $p = 0.003$) and level with the single tool-using agent

(24.0% each, $p = 1.0$), while using more tokens than either (9,182 per test against 8,495 and 4,121). These head-to-head comparisons mix G2 and G6, because s3 targets consensus smells and both baselines target every tsDetect smell. With matched targets (s3-G2), the adaptive system reaches 39.3%: clearly above b2 (24.0%, Holm $p < 0.001$) and 4 points above b1, a difference that is not significant ($p = 0.24$; interval -1.3 to 9.3). Inside s3, replacing the adaptive policy by b1's fixed script changes accepted-and-safe from 24.0% to 26.0% ($p = 0.55$) and cuts tokens per test by 27%.

D. Gap-resolution table

Table IV is produced by `gap_table.py` from `results/` and is the same table as in the repository README. The "Addressed?" column is our judgement from the paired tests above.

VIII. Ablation and Comparative Analysis

Table V switches the components off one at a time (E5): -G1 keeps PIT but never rejects on it, -G2 targets every tsDetect smell, -G6 replaces the adaptive policy with b1's fixed retry script (memory is off with it), memory turns off only the memory, and minus_all does all of this at once. PIT measures in every variant, so kill-set loss is comparable across rows. Figure 2 plots accepted-and-safe per variant.

The kill-set gate is the only component whose removal makes outputs less safe: without it the loss rate goes from 0 to 8.0%, and minus_all loses mutants in 3.3% of its accepted outputs. Removing consensus (-G2) lowers acceptance by 7.7 points and raises rule-i reduction by 13.1 points and accepted-and-safe by 13.5 points, at 2,046 more tokens per test. Of all the components, consensus moves the headline metric the most, and it moves it in the wrong direction for the reason given in RQ2. The fixed policy (-G6) passes more tests (84.7% against 74.9%) but accepts slightly fewer, and it needs 2.0 LLM calls per test instead of 3.0. Memory has almost no effect on acceptance (-0.4 points) and costs 4.7 points of rule-ii reduction when it is off. Within-test memory still matters in the trajectories: it rejects a proposal identical to an earlier failed one before it is verified, which shows up as the "malformed reply" category below.

Sensitivity. Table VI varies the retry cap of s3 on 20 dev tests without PIT, so the test split is not touched. Acceptance rises from 75% with no retries to 90% with one and 95% with four, and tokens per test grow from 4,317 to 6,698. Most of the gain comes from the first retry; the cap of 2 used in all reported runs sits on the flat part of the curve.

Efficiency and reliability. Table VII reports tokens split into prompt and completion, LLM and tool calls, wall-clock and PIT time per test, and retry and recovery rates. Cost is $0 on the Cerebras free tier, so tokens stand in for cost; Figure 3 plots tokens against accepted-and-safe. s3 makes 3.0 LLM calls and 2.8 tool calls per test and spends 104 ± 14 s per test, 71 s of it in PIT. It retries 60% of the tests and recovers 28.6 ± 5.9% of those, more than b1 (18.5%) and less than b2 (34.0%). Wall-clock times are only indicative: during the reported runs several configurations ran in parallel on one machine and shared its CPU (Section V).

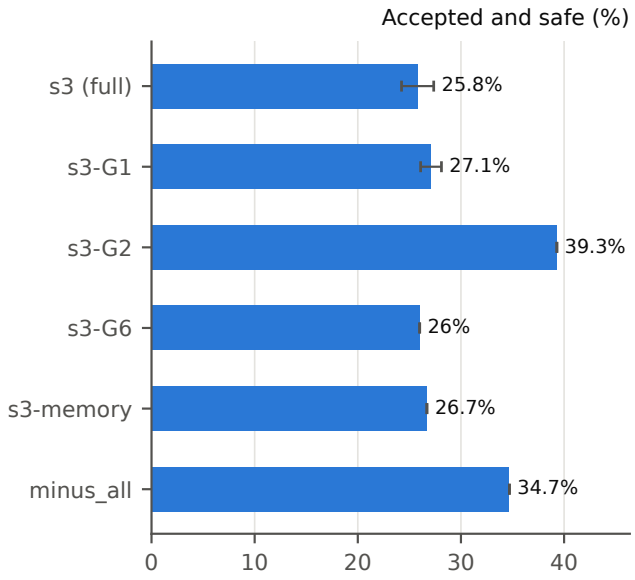


Fig. 2. Accepted-and-safe rate of the full system and each ablation.

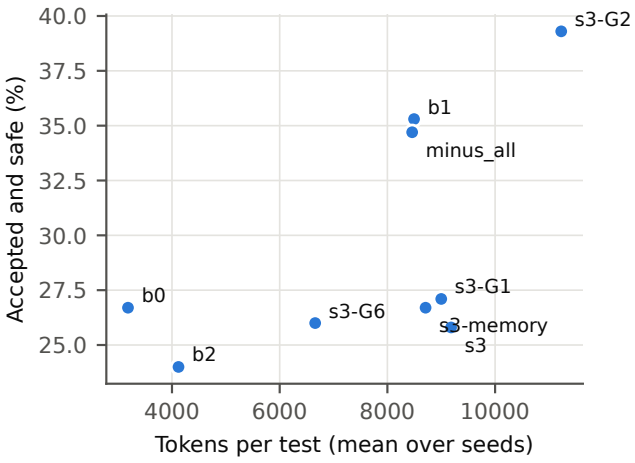


Fig. 3. Tokens per test against accepted-and-safe rate, one point per configuration.

IX. Failure Analysis

Each test that s3 did not accept gets exactly one category, assigned from its final verification report by `failures.py`; the first matching rule wins: infrastructure error, budget exhausted, malformed reply, assertion dropped or empty test, API not in the project, missing symbol or import, other compile error, fixture clobbered by an extracted setup, semantic change (the test now fails), kill-set loss, and smell not removed or new smell. Table VIII and Figure 4 count them over the three s3 seeds, each (test, seed) pair once.

TABLE IV
GAP-RESOLUTION TABLE (IDENTICAL TO THE README). BASE-PAPER CELLS STATE WHETHER THE NUMBER WAS REPRODUCED BY US OR REPORTED IN [1].

Gap	Method / component	Metric	Base paper result	Our result (S3)	Addressed?
G1	Kill-set gate (`killset_gate.py`)	Accepted outputs losing a mutant (%)	1.2 (b0, reproduced); 3.4 (b0replay, reproduced)	0	Fully
G2	Consensus detector, tsDetect + JNose (`consensus.py`)	Smell reduction, rule ii (%)	61.7 (b0, reproduced); 54.6 (b0replay, reproduced); rule i: 86 / 89 / 91 (reported in [1])	52.47 ± 1.7	Partially
G6	Adaptive Supervisor + memory (`agents/supervisor.py`)	Accepted and safe (%)	26.7 (b0, reproduced); 11.8 (b0replay, reproduced)	25.8 ± 1.56	Not

TABLE V
ABLATION (E5): EACH VARIANT'S CHANGE FROM THE FULL SYSTEM (E3). POSITIVE VALUES ARE HIGHER WITHOUT THE COMPONENT.

Config	Accepted %	Passes %	Red. i %	Red. ii %	Lose mut. %	Acc.+safe %	Tok/test
full (E3)	52.43 ± 1.03	74.9 ± 2.03	46.07 ± 1.6	52.47 ± 1.7	0	25.8 ± 1.56	9182.33 ± 145.89
Δ s3-G1	2.7	0.63	1.86	-1.24	8	1.3	-182.33
Δ s3-G2	-7.73	-8.2	13.13	8.33	0	13.5	2045.67
Δ s3-G6	-1.13	9.8	7.23	-1.87	0	0.2	-2523.33
Δ s3-memory	-0.43	-0.2	-0.47	-4.67	0	0.9	-473.33
Δ minus_all	-10.43	-7.6	16.03	7.83	3.3	8.9	-722.33

TABLE VI
SENSITIVITY TO THE RETRY CAP (ES, 20 DEV TESTS, NO PIT).

System	Accepted %	Passes %	Red. i %	Red. ii %	Lose mut. %	Acc.+safe %	Tok/test
s3-retries0	75	85	79.3	75.9	–	–	4317
s3-retries1	90	95	86.2	82.8	–	–	5535
s3-retries2	90	90	75.9	84.6	–	–	6276
s3-retries4	95	95	79.3	88.5	–	–	6698

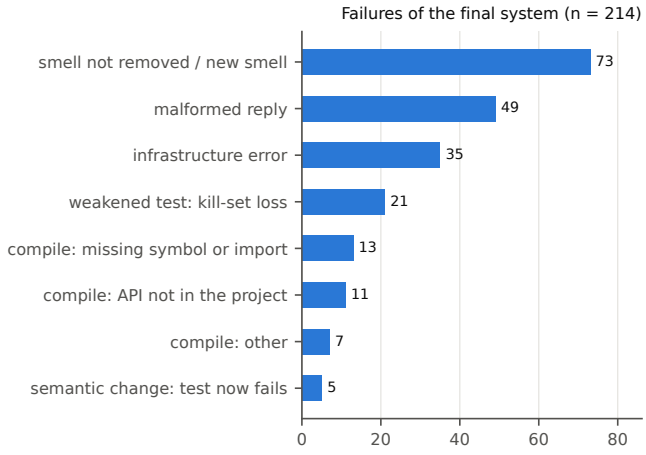


Fig. 4. Failure-mode distribution of the full system.

Of 450 pairs, 214 were not accepted. A targeted smell that survived every attempt is the largest category (73 pairs, 34.1% of failures), followed by malformed replies (49, 22.9%), infrastructure errors (35, 16.4%) and kill-set loss (21, 9.8%). Compile failures add up to 31 pairs (14.5%), and only 5 failures (2.3%) are tests that compile but no longer pass. Almost all malformed replies (46 of 49) are repeats: at temperature 0.7 the model often returns a refactoring identical to one the verifier already rejected, and memory refuses to verify it again. All 35 infrastructure errors are PIT runs that produced no report (21 on Gson, 14 on Jsoup), and the same classes fail in every system: Jsoup's `ConnectTest` needs network access that PIT cannot provide, and some Gson classes fail PIT's own compilation step. We report them as errors instead of dropping the tests.

Trajectories. We walk through two accepted and two rejected tests from `logs/E3_s3_s1` (exported with `failures.py --case`).

Success 1, clean accept (`commons-cli: OptionGroupTest# testTwoOptionsFromDifferentGroup`). tsDetect reports ExceptionHandling for the `throws Exception` clause and JNose reports nothing, so the Detection agent falls back to the tsDetect label. The Planner proposes removing the clause and wrapping the body; the Refactor agent wraps the six assertions in JUnit 5's `assertDoesNotThrow`. The Verifier finds that the class compiles and passes, the smell is gone, and all 63 mutants killed by the original are still killed; the Supervisor accepts after one attempt (65 s, 12 s of it in PIT).

Success 2, accepted after a retry (`gson: GsonTest# testNewJsonReader_Default`). The first attempt makes the same mistake as the published UTRefactor outputs for Gson: it imports `org.junit.jupiter.api`, which does not exist in this JUnit 4 project, and compilation fails. The

TABLE VII
EFFICIENCY AND RELIABILITY PER CONFIGURATION (E6), MEANS OVER SEEDS. RETRIED: SHARE OF TESTS WITH AT LEAST ONE RETRY; RECOVERY: ACCEPTED AMONG THE RETRIED.

Config	Tok/test	Prompt	Compl.	LLM calls	Tool calls	s/test	PIT s	Retried %	Recovery %	Cost $
E1/b0replay	0	0	0	0	0.96	9.26	5.58	0	–	0
E2/b0	3182.67	1959.59	1223.08	1	0.97	3.54	0	0	–	0
E2/single	1799.35	802.45	996.89	1	1	15.84	0	0	–	0
E2/b1	8494.95	5323.39	3171.55	2.25	2.18	88.96	60.74	68.67	18.45	0
E2/b2	4121.37	2840.74	1280.63	1.31	0.96	56.92	15.3	31.33	34.04	0
E3/s3	9182.34 ± 146.06	5899.65 ± 132.92	3282.69 ± 41.08	3.03 ± 0.05	2.84 ± 0.06	103.73 ± 13.81	71.1 ± 10.02	60 ± 5.03	28.62 ± 5.86	0
E5/s3-G1	8999.84 ± 140.47	5814.35 ± 56.5	3185.49 ± 91.02	3.02 ± 0.03	2.86 ± 0.05	106.92 ± 14.74	73.64 ± 13.85	60.67 ± 0.67	33.3 ± 4.59	0
E5/s3-G2	11228.3	7331.52	3896.78	3.27	3.11	107.54	69.83	73.33	28.18	0
E5/s3-G6	6659.18	4074.17	2585.01	2.01	2.92	58.16	39.25	58	20.69	0
E5/s3-memory	8709.44	5510.67	3198.77	3.01	2.91	101.01	71.86	59.33	32.58	0
E5/minus_all	8459.77	5312.26	3147.51	2.25	3.19	30.66	6.1	69.33	20.19	0

TABLE VIII
FAILURE MODES OF THE FULL SYSTEM (E3, THREE SEEDS, 450 TEST–SEED PAIRS).

Failure mode	Count	% of failures	% of tests
infrastructure error	35	16.4	7.8
malformed reply	49	22.9	10.9
compile: API not in the project	11	5.1	2.4
compile: missing symbol or import	13	6.1	2.9
compile: other	7	3.3	1.6
semantic change: test now fails	5	2.3	1.1
weakened test: kill-set loss	21	9.8	4.7
smell not removed / new smell	73	34.1	16.2
Total	214	100	47.6

Supervisor retries with the compiler message. The second reply defines a private `assertDoesNotThrow` helper on top of JUnit 4's `fail`, compiles, passes, keeps all 18 mutants and removes the smell. This is the kind of case a one-pass method cannot recover from.

Failure 1, smell not removed (`gson: JsonPrimitiveTest#testByteEqualsInteger`). Both detectors label AssertionRoulette and EagerTest. The first attempt splits the test into an equality test and a hash-code test with a shared helper; it compiles, passes and keeps all 6 mutants, but tsDetect still reports EagerTest while JNose reports nothing. The Supervisor narrows the target to EagerTest, then switches from the LLM plan to the knowledge-base rule; both attempts keep the same two-test split, and the test is escalated. Root cause: the remaining failure is a disagreement between the detectors about a refactoring that is already reasonable, not a defect the model can repair. A fix is a stop rule: when one detector is clean and the output is unchanged after a structural retry, escalate at once and report the disagreement, instead of spending two more LLM calls.

Failure 2, kill-set loss (`gson: JsonWriterTest#testDoubles`). The original writes many doubles into one array and checks the whole string with a single assertion. Attempt 1 splits it into several tests with nine assertions in total; it compiles and passes but kills 21 of the original 24 mutants, so the gate rejects it with the list of lost mutants. Attempt 2 restores all 24 but leaves EagerTest. Narrowed to EagerTest, attempt 3 removes the smell and loses the same three mutants again, and the test is escalated. Root cause: in this test the smell and the mutants pull in opposite directions; the single whole-array assertion is what kills the three mutants, and splitting it away is what removes EagerTest. Without the gate, attempt 1 would have been accepted as a successful refactoring. A fix is to keep one whole-array assertion in a separate test next to the split ones, which the Planner could propose when the gate's report names the lost mutants.

X. LIMITATIONS AND THREATS TO VALIDITY

Internal. LLM output is not deterministic on a hosted model, even with a fixed seed, and our results depend on the prompts and on the retry and token caps, which we tuned on the dev split only. Three seeds bound this variance for the full system and its -G1 variant; every other row is a single seed and has no variance estimate, so differences of a few points between single-seed rows (for example s3-G2 against b1) should not be read as real. The reported runs did not happen in one sitting. The free tier allows 5 requests per minute, 150 per hour and about one million tokens per day per key, so the runs were spread over several API keys and resumed many times; each configuration still ran in its own copy of the repository at the same commit, with one key at a time, and a resumed run only skips tests whose result was already saved. When a key ran out, the provider's retry delay sometimes stalled a run for hours until the key was swapped. Between resumptions the hosted model could in principle have changed, which we cannot detect. PIT's default behaviour counts a mutant that makes the test time out as killed, and 4 to 16 tests per configuration end in a tool error (mostly PIT on Jsoup's network tests and on a few Gson classes); these count as not accepted.

External. We use three Java projects (Commons-cli on JUnit 5, Gson and Jsoup on JUnit 4) and 150 held-out tests, with the tsDetect smell set. That supports no claim about other languages, frameworks or larger suites. All systems use one model, `gpt-oss-120b`. A stronger model might make fewer compile errors and change the balance between retrying and planning; a weaker one might make the gate reject far more often.

Construct. The PIT kill set is a proxy for test strength: a weaker assertion can still kill the same mutants, and mutants outside PIT's default mutators are invisible to it. Tests with

an empty baseline kill set are reported as unknown, not as safe. "Smell removed" depends on the detector: tsDetect and JNose disagree on ExceptionHandling and DuplicateAssert, and the trajectories show escalations caused by such disagreements alone. Our aggregate rule-ii reduction is not directly comparable to rule i, because rule ii is computed only over outputs that JNose could analyse. The accepted-and-safe rate checks all smells labelled in [1]; this choice makes consensus targeting look worse than a metric over targeted smells would, and we report both views. Rule iii (the manual sample) was not completed. JaCoCo coverage, planned in Phase 1, was not used. Finally, our evaluation set has 484 tests rather than the 593 of [1], because only these match the pinned project commits, and Gson is compiled against its own test dependencies, which [1] evidently did not do.

XI. Conclusion

We built a multi-agent pipeline that accepts a test-smell refactoring only when it compiles, passes, removes the targeted smells and keeps every mutant the original test killed, and compared it with UTRefactor and four baselines on 150 held-out tests. **G1** is addressed: with the kill-set gate no accepted output lost a mutant, against $8.0 \pm 0.5\%$ without it, at no measurable token cost and 2.7 points of acceptance. UTRefactor's own published outputs lose mutants in 3.4% of the cases our verifier could check. **G2** is partly addressed: the measurement side holds, since the two detectors disagree sharply on some smells ($\kappa = 0.02$ for ExceptionHandling) and rule ii removes credit for AssertionRoulette and MagicNumberTest that rule i gives. Using consensus to choose targets, however, lowered the accepted-and-safe rate by 15 points. **G6** is not addressed: the adaptive Supervisor did not beat a fixed retry script, either as the b1 baseline or inside our own system, and it used more tokens; it did beat a single tool-using agent when both targeted the same smells.

Three lessons carry over to similar systems. Acceptance should be decided by tools: the kill-set gate was the only component whose removal made outputs less safe, although PIT also accounts for most of the runtime (71 of 104 s per test). The baseline kill matrix has to be computed before the patch is written; our Phase 1 pilot measured the refactored code by mistake. And a reproduction should report the build it used: 131 of the 238 published Gson refactorings call a JUnit 5 method that Gson's JUnit 4 test setup does not have. For future work, the trajectories point to two concrete changes: a stop rule that escalates when only one detector still reports a smell, and a planner that keeps a whole-object assertion next to split tests when the gate names lost mutants. Beyond that, more projects and a second model family would show whether the G6 result is specific to this setting.

References

[1] Y. Gao, X. Hu, X. Yang, and X. Xia, "Automated unit test refactoring," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2025, arXiv:2409.16739.

[2] V. Garousi and B. Küçük, "Smells in software test code: A survey of knowledge in industry and academia," *Journal of Systems and Software*, vol. 138, 2018.

[3] M. Tufano, F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, and D. Poshyvanyk, "An empirical investigation into the nature of test smells," in *ASE*, 2016.

[4] G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley, "Are test smells really harmful? an empirical study," *Empirical Software Engineering*, vol. 20, no. 4, 2015.

[5] A. Peruma, K. Almalki, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba, "tsdetect: An open source test smells detection tool," in *ESEC/FSE (Tool Demonstrations)*, 2020.

[6] T. Virgínio, L. Martins, L. Soares, R. Santana, H. Costa, and I. Machado, "Jnose: Java test smell detector," in *SBES*, 2020.

[7] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, 1960.

[8] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, 1977.

[9] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *FSE*, 2014.

[10] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "Pit: A practical mutation testing tool for java," in *ISSTA*, 2016.

[11] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Software Engineering*, vol. 37, no. 5, 2011.

[12] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*, 2019, vol. 112.

[13] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Software Engineering*, vol. 41, no. 5, 2015.

[14] G. Fraser and A. Arcuri, "Evosuite: Automatic test suite generation for object-oriented software," in *ESEC/FSE*, 2011.

[15] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Software Engineering*, vol. 50, no. 1, 2024.

[16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *ICLR*, 2023.

[17] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," in *NeurIPS*, 2023.

[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *NeurIPS*, 2023.

[19] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *arXiv preprint*, 2022.

[20] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," *arXiv preprint*, 2023.

[21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *ICLR*, 2024.

[22] J. Yang *et al.*, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint*, 2024.

[23] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint*, 2021.

[24] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, 1947.

[25] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, 1945.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, 1927.